

FlexiGrid AI

Evidence-grounded household energy planning with RAG, agents, MCP tools and Elia Open Data

PROJECT THESIS Use retrieval and structured generation to explain bounded household missions, while keeping energy scheduling and safety constraints deterministic, testable and reproducible.

3 + 1 advanced techniques RAG, agents, MCP, transformer	9/9 fixture configurations hard constraints passed	18% fixture improvement cost and grid exposure vs baseline
--	---	---

AUTHORS

Nima Asadi and project partner

Team of two • August 2026 • Howest

Executive summary

FlexiGrid AI is a household energy flexibility copilot for Belgium. A user states a goal such as charging an electric vehicle, pre-heating a home and completing appliances before a deadline. The system retrieves the relevant device and household constraints, calls typed grid-data tools, computes a feasible 24-hour schedule and generates a cited explanation. The prototype intentionally separates language generation from physical feasibility: a deterministic validator, not the language model, decides whether a plan is safe to display.

RECOMMENDATION Submit FlexiGrid AI as the project. It demonstrates several advanced generative AI techniques in one coherent workflow, uses a credible Belgian data source, has measurable failure criteria and fits a 10-minute demonstration.

Problem definition

Households with an EV, heat pump and flexible appliances face competing objectives: finish tasks before deadlines, maintain comfort, avoid a connection-capacity peak and shift consumption away from grid-stress periods. Existing energy dashboards expose curves but still require the resident to translate those curves into a schedule. A generic chatbot is not sufficient because fluent text can violate power limits or confuse market prices with consumer tariffs.

Target user and success definition

The target user is a Belgian household or energy-coaching professional who needs an understandable next-day plan. Success is not defined as 'helpful text'. A successful response must satisfy every time window, stay at or below the configured 4.6 kW controllable-load cap, cite the evidence used, keep retail pricing separate from Elia imbalance-market concepts, and produce the same result when the frozen fixture and parameters are unchanged.

Project objectives

- Represent three natural-language household missions as typed, reproducible planning requests.
- Ground all device, comfort and data-source claims in retrieved chunks.
- Expose Elia, household and optimizer functions through MCP-style tools.
- Generate explanations only after the schedule passes deterministic validation.
- Evaluate retrieval, tool use, planning correctness and generation separately.

1. Data selection and pipeline

Elia, Belgium's transmission system operator, publishes open datasets through an Opendatasoft API. Three datasets define the grid-data contract for FlexiGrid AI. The live adapter preserves raw records and retrieval timestamps. The submitted planner uses a clearly labelled representative 0-100 stress fixture; deriving that signal from changing live schemas remains future pipeline work. This keeps the demo reproducible and prevents synthetic values from being presented as observations.

Dataset	Official content	Role in FlexiGrid
ods002	Measured and forecast total Belgian load	Intended peak/load-pressure feature
ods086	Intraday, day-ahead and week-ahead wind forecast	Intended renewable-opportunity feature
ods201	Actual generation aggregated by fuel type	Generation-mix context and future carbon proxy

Official sources: [ods002](#) | [ods086](#) | [ods201](#)

Data contract

1. Fetch up to 100 recent records per supported dataset from Elia's v2.1 records endpoint.
2. Preserve dataset identifier, raw records and an ISO retrieval timestamp for reproducibility.
3. Return raw live records when explicitly enabled; otherwise use the frozen fixture.
4. Keep the representative 0-100 stress fixture labelled as demo data until a schema-tested live normalization stage is implemented.

IMPORTANT SEMANTIC GUARDRAIL Elia imbalance prices are settlement signals for balance responsible parties, not a household retail tariff. FlexiGrid therefore uses a separate labelled retail-tariff fixture for cost optimization.

Privacy and EU constraints

The submitted prototype uses no personal smart-meter records and stores no user identity. A production pilot would require explicit household consent, data minimization, retention limits, device authentication, audit logs and a controller/processor assessment under GDPR. The prototype remains advisory: it does not send commands to physical devices.

2. Technical architecture

FlexiGrid AI control flow



Figure 1. The language model is bounded by retrieved evidence, typed tools and a deterministic validator.

Retrieval-Augmented Generation

The retriever indexes short chunks from device manuals, a household comfort policy, the connection-capacity rule, tariff semantics and Elia dataset descriptions. The current baseline uses transparent lexical overlap and returns top-k = 4 chunks. This is deliberately small and reproducible; the next scaling step is a hybrid BM25 plus embedding retriever with a reranker. Retrieved chunk IDs become the only citation IDs the generator is allowed to return.

Agent workflow

The workflow separates planning and criticism. The planner receives the typed tasks and objective. The critic independently checks the output against time windows and hourly power. Only a valid plan is passed to the explanation model. If validation fails, the plan is rejected rather than repaired through persuasive prompting.

Model Context Protocol

MCP tool	Input	Output / responsibility
get_elia_grid_snapshot	use_live: bool	Live records or frozen fixture with provenance
retrieve_household_evidence	query, top_k	Ranked chunks with stable citation IDs
get_demo_device_constraints	none	Typed power, duration and time windows
optimize_household_plan	objective	Schedule, metrics and validator result

Transformer model and structured output

When an API key and model are configured, the backend calls the OpenAI Responses API with a Pydantic Explanation schema. The response contains a concise summary, two to four rationale statements, citation IDs and a limitation. The backend rejects any citation ID that is not present in the retrieved evidence. Without a key, the same endpoint returns a deterministic cited explanation, keeping the prototype fully demonstrable offline.

3. Planning algorithm and design choices

Tasks are modelled as power, duration, earliest start, latest end and source ID. For each task, the optimizer enumerates feasible start hours and scores them using a weighted combination of normalized retail tariff and grid stress. A depth-first exhaustive search explores joint assignments and prunes branches whose partial score already exceeds the best complete solution.

$$score(h) = w \cdot normalized\ tariff(h) + (1 - w) \cdot grid\ stress(h)$$

VALIDATION FINDING The first greedy implementation could place the EV in a locally attractive slot and leave no feasible heat-pump window. Automated tests exposed this defect. The algorithm was replaced with joint constrained search; all nine scenario-objective combinations now pass.

Why not let the LLM schedule directly?

The current model layer explains an already validated bounded mission; it does not claim arbitrary intent extraction. Exact power and deadline constraints are represented as code. This hybrid design improves reliability, makes failures reproducible and gives the exam team a clear baseline. It also reduces model cost because the optimizer does not consume tokens.

Why not fine-tune?

Fine-tuning is not justified for this deadline or data volume. The changing knowledge consists mainly of manuals, household preferences and public grid feeds, which are better handled through retrieval and tools. Fine-tuning would be considered only after collecting a sizeable labelled set of intent-to-constraint examples and showing that prompting plus retrieval remains the bottleneck.

Complexity and scalability

Exhaustive search is appropriate for the four-device prototype but grows combinatorially. A production version should replace it with a mixed-integer linear program or constraint-programming solver, preserve the same tool contract and compare solution quality and latency against the exhaustive oracle on small fixtures.

4. Prototype implementation

The submission contains a polished responsive interface and a separate Python technical backend. The interface demonstrates the complete user flow without external secrets; the backend shows the real RAG, Elia, MCP, optimizer and optional model integration that would power a production API.

Layer	Technology	Purpose
Interface	React 19, TypeScript, Vinext	Scenario selection, schedule visualization, evidence and tool trace
API	FastAPI, Pydantic	Typed plan and grid-snapshot endpoints
GenAI	OpenAI Responses API	Schema-constrained cited explanation
MCP	Python MCP SDK / FastMCP	Interoperable typed energy tools
Data	Elia v2.1 API + JSON fixture	Belgian load, wind and generation context
Tests	Node test + unittest	Rendered UI, optimizer, retrieval and reproducibility

Primary demonstration flow

1. Choose Morning, Grid friendly or Peak shield, then select Balanced, Cost or Grid support.
2. Run the agent plan and watch retrieval, MCP tool calls, optimization and criticism progress.
3. Compare optimized and earliest-start schedules on the 24-hour chart.
4. Inspect cost, grid-stress, hard-constraint and evidence metrics.
5. Open Evaluation to defend the measured results, then Architecture to explain design choices.

Reproducibility

The public interface requires no API key. The repository README gives exact commands for the UI, FastAPI service, MCP server, live snapshot script and both test suites. Dependency ranges are constrained, the fixture is versioned and every displayed benchmark number is calculated from deterministic runs rather than copied from a model response.

5. Evaluation

Evaluation is split by subsystem. This prevents a strong-looking natural-language answer from hiding retrieval, tool or optimization failures. The current automated suite covers nine combinations in the browser-side engine and three objective modes in the Python engine.

Fixture day: flexible load moves into lower-pressure hours

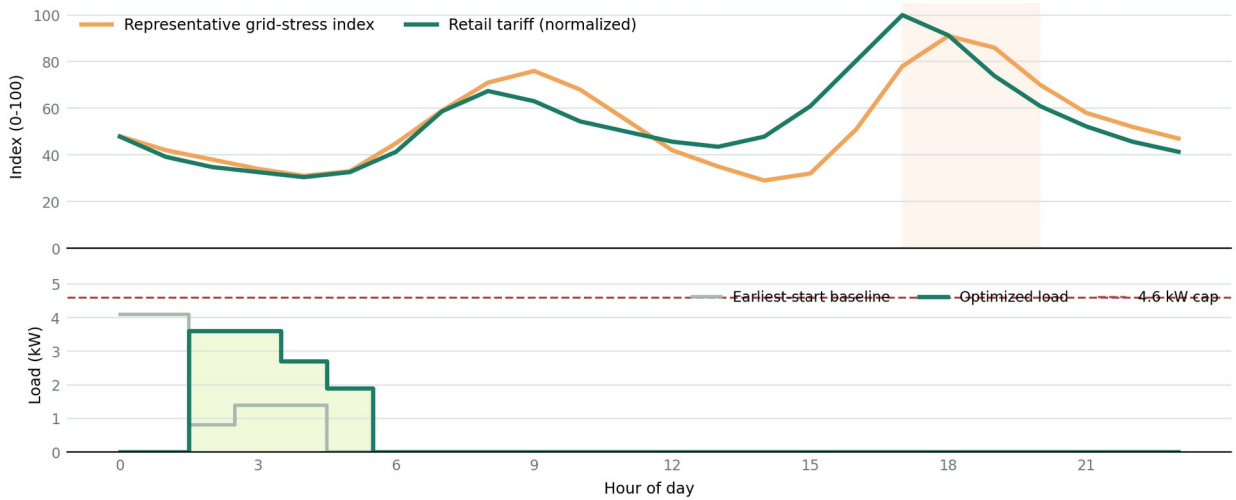


Figure 2. Reproducible fixture: retail tariff and representative grid-stress index above; baseline and optimized controllable load below.

Automated results

Metric	Result	Basis	Acceptance criterion
Constraint pass rate	100% (9/9)	3 scenarios × 3 objectives	100%
Cost non-regression	100% (9/9)	Compared with earliest-start baseline	≥ 90%
Morning fixture peak	3.6 kW	Hour-by-hour validator	≤ 4.6 kW
Morning fixture cost	€1.79	Frozen retail tariff	Lower than €2.17 baseline
Determinism	Pass	Repeated plan deep equality	Identical output
EV retrieval	Rank 1	Labelled lexical query	EV manual first

These numbers validate the deterministic prototype, not generalize an LLM benchmark. Before claiming model quality, the team should label at least 30 unseen prompts and report retrieval recall@4, citation precision, tool-selection accuracy, groundedness and task success with bootstrap confidence intervals.

Baselines and ablations

- Baseline A: earliest feasible start, which satisfies constraints but ignores signals.
- Baseline B: LLM-only scheduling, evaluated for constraint violations and invented citations.
- Ablation 1: remove RAG to measure evidence-groundedness loss.
- Ablation 2: remove the critic to measure invalid-plan display rate.
- Ablation 3: replace exhaustive search with greedy placement to reproduce the discovered failure.

6. Limitations, risk and responsible use

Risk	Current control	Production requirement
Hallucinated rationale	Citation allow-list and structured output	Groundedness grader + human review
Unsafe schedule	Deterministic hard-constraint validator	Device-specific safety envelope and fail-safe
Stale or synthetic grid signal	Timestamped raw adapter + explicit fixture label	Schema-tested live derivation, freshness SLO and monitoring
Tariff confusion	Retail and imbalance concepts separated	Supplier-specific billing contract tests
Privacy leakage	No personal meter data in prototype	Consent, minimization, encryption and retention policy
Search scalability	Exhaustive search on four tasks	MILP/CP-SAT optimizer with time budget

Team collaboration

The work should be divided without creating knowledge silos. Both students must review the complete code path and rehearse the same demo. A practical division is shown below; names can be assigned in the repository issue board while responsibilities remain shared.

Workstream	Team member A	Team member B	Shared verification
Data and backend	Elia adapter, optimizer, MCP tools	API integration and RAG corpus	Pair review + backend test run
Interface and evaluation	Dashboard and chart	Fixture labels and evaluation harness	UI walkthrough + benchmark review
Documentation and defense	Methods and implementation	Results, limitations and demo script	Cross-question rehearsal

Conclusion

FlexiGrid AI is technically appropriate for the assignment because the advanced techniques are connected to a concrete workflow. RAG supplies changeable evidence, MCP separates tool contracts, structured generation explains only validated results and deterministic optimization guarantees feasibility. The bounded mission scope and labelled fixture keep current claims honest. The Belgian Elia integration makes the project locally relevant, while the automated acceptance suite makes it reproducible under exam conditions.

References

- [Elia Open Data - measured and forecast total load \(ods002\)](#)
- [Elia Open Data - wind power forecast \(ods086\)](#)
- [Elia Open Data - total generation by fuel type \(ods201\)](#)
- [OpenAI API - Structured model outputs](#)
- [Model Context Protocol - specification](#)